

Informe Tecnico

Evaluacion Final Transversal (EFT)

Migracion del sistema legacy del Banco XYZ
a microservicios con Spring Cloud, Kafka y Docker

PBY2203 — Desarrollo Backend III
DuocUC Online

Sergio Mascareno
Octubre 2026

Repositorio: <https://github.com/Zersource/banco-xyz-bff>
Rama: eft-final

Indice

1. Resumen ejecutivo
2. Contexto y requerimientos de negocio
3. Arquitectura propuesta
4. Procesos batch (Spring Batch)
5. Patron BFF por canal
6. Microservicios, Kafka y resiliencia
7. Seguridad con Spring Cloud Security (OAuth2)
8. Contenerizacion y escalabilidad con Docker
9. Evidencia de ejecucion
10. Desafios y soluciones
11. Propuestas de mejora
12. Conclusiones

1. Resumen ejecutivo

Este informe documenta la migración del sistema legacy del Banco XYZ a una arquitectura de microservicios. El proyecto abarca cinco ejes de migración: la conversión de procesos batch a Spring Batch con manejo de errores y paralelismo; la separación del monolito en microservicios independientes con Spring Cloud (Eureka, Config Server); la implementación del patrón BFF para tres canales (web, móvil, cajero); la seguridad con OAuth2 usando Spring Authorization Server (client_credentials, JWT, JWKS); y la mensajería asíncrona con Apache Kafka para una saga de transferencias con compensación.

Todo el ecosistema — 10 aplicaciones Java, un broker Kafka y un balanceador nginx — se conteneriza con Docker y Docker Compose, con builds multi-stage, healthchecks, arranque ordenado y escalabilidad horizontal demostrada en los BFF. El stack completo se verificó con una prueba end-to-end automatizada de 95 verificaciones, ejecutada tanto en local como en Docker, incluyendo un clon limpio del repositorio.

2. Contexto y requerimientos de negocio

El Banco XYZ opera un sistema legacy monolítico con procesos batch basados en archivos planos, sin separación de canales y sin tolerancia a fallos. La gerencia de TI requiere migrar a una arquitectura moderna que cumpla tres requerimientos de negocio:

- **Continuidad operacional:** los procesos batch de transacciones diarias, intereses mensuales y estados de cuenta anuales deben seguir funcionando sin pérdida de datos, con capacidad de recuperación ante fallos transitorios y permanentes.
- **Experiencia diferenciada por canal:** cada canal de atención (web, móvil, cajero) necesita una fachada optimizada que entregue solo los datos pertinentes, con seguridad independiente y tolerancia a fallos cuando un servicio backend no este disponible.
- **Transferencias en tiempo real:** el sistema de transferencias entre cuentas debe ser asíncrono, garantizar consistencia eventual (débito-credito atómico) y compensar automáticamente ante fallos, notificando a los titulares.

Estos tres requerimientos justifican la arquitectura propuesta: Spring Batch para el procesamiento legacy, BFF con Circuit Breaker para los canales, y una saga coreografiada sobre Kafka para las transferencias.

3. Arquitectura propuesta

Diseñe la arquitectura como un ecosistema de 10 aplicaciones Spring Boot independientes, cada una con su propio módulo Maven, su Dockerfile y su configuración en el Config Server. No use un POM padre: cada módulo se compila y despliega de forma autónoma.

3.1 Componentes de infraestructura

Componente	Puerto	Tecnología	Función
config-server	8888	Spring Cloud Config (native)	Configuración centralizada para todos los servicios

Componente	Puerto	Tecnologia	Funcion
eureka-server	8761	Spring Cloud Netflix Eureka	Registro y descubrimiento de servicios
auth-server	9000	Spring Authorization Server 1.3.2	Emision de tokens OAuth2, JWKS publico
kafka	9094	Apache Kafka 3.7 (KRaft)	Broker de mensajeria para la saga

3.2 Microservicios de negocio

Servicio	Puerto	Responsabilidad
cuentas	8084	Cuentas, saldos, movimientos. Ejecuta debito, credito y compensacion de la saga
clientes	8085	Datos del titular de cada cuenta. Notifica transferencias completadas
pagos	8086	Recibe transferencias, las persiste en H2 y orquesta la saga via Kafka

3.3 BFF (Backend for Frontend)

BFF	Puerto	Canal	Datos expuestos
bff-web	8081	web	Cuentas completas, transacciones, transferencias
bff-movil	8082	movil	Cuenta resumida (payload liviano)
bff-cajero	8083	cajero	Solo saldo y retiro

3.4 Diagrama de flujo

```

cliente → token → bff-web / bff-movil / bff-cajero
      ↓ (Eureka + RestClient + Circuit Breaker)
cuentas  clientes  pagos
      ↓           ↓
      Kafka (saga de transferencias)

```

Todos los servicios se registran en Eureka y obtienen su configuracion del Config Server. Los BFF resuelven los servicios por nombre (no por host:puerto fijo) y protegen cada llamada con Circuit Breaker. Los microservicios de negocio se comunican entre si exclusivamente por Kafka.

4. Procesos batch (Spring Batch)

Implemente tres jobs de Spring Batch que procesan los datos legacy del banco, cada uno con su propio Reader, Processor y Writer, validacion de registros, manejo de skip para datos invalidos y retry ante fallos transitorios.

4.1 Jobs implementados

Job	Descripcion	Resultado
dailyTransactionsJob	Carga transacciones diarias, filtra anomalias	387 escritos, 599 anomalias
monthlyInterestJob	Calcula intereses mensuales sobre saldos	50 escritos, 601 anomalias
annualStatementJob	Genera estados de cuenta anuales	614 escritos, 538 anomalias

4.2 Manejo de errores

Configure tres niveles de proteccion ante fallos:

- **Skip:** los registros que no pasan la validacion de negocio (formato invalido, campos faltantes, montos negativos) se omiten y se registran en el SkipListener para auditoria. El job continua sin detenerse.
- **Retry:** ante fallos transitorios (TransientDataAccessException), el step reintenta hasta 3 veces antes de fallar. Esto cubre interrupciones momentaneas de I/O.
- **Restart con Docker:** el contenedor batch tiene restart_policy con max_attempts: 3. Si el job falla (exit code 1), Docker lo reinicia automaticamente. Probado con falla permanente (4 intentos, exit 1, RestartCount 3) y falla transitoria (el CSV aparece en el segundo intento, exit 0, RestartCount 1).

4.3 Paralelismo y escalabilidad

El dailyTransactionsJob usa particionamiento con 3 workers: el Partitioner divide el archivo de entrada en 3 segmentos y cada worker (partition0, partition1, partition2) procesa su rango de forma concurrente con un TaskExecutor de pool size 3. Los otros dos jobs procesan secuencialmente porque su logica de negocio (interes acumulado, estado de cuenta consolidado) requiere acceso al dataset completo.

5. Patron BFF por canal

Implemente el patron Backend for Frontend con tres fachadas independientes, cada una optimizada para su canal de atencion. La diferencia entre canales no es solo de seguridad (scope OAuth2) sino de contenido: cada BFF expone endpoints distintos con payloads de peso diferente.

5.1 Diferenciacion por canal

Canal	BFF	Endpoints	Caracteristicas
Web	bff-web (8081)	/api/web/cuentas, /api/web/transacciones, /transferencias	Datos completos: saldo, titular, movimientos, transacciones. Unico canal con transferencias
Movil	bff-movil (8082)	/api/movil/cuentas/{id}	Respuesta liviana: solo id, saldo y titular. Payload reducido para conexiones moviles
Cajero	bff-cajero (8083)	/api/cajero/cuentas/{id}/saldo, .../retiro	Operaciones basicas: consulta de saldo y retiro. Sin acceso a transacciones ni transferencias

5.2 Rendimiento por canal

El script e2e verifica que el peso del payload es coherente con el proposito de cada canal. El BFF web devuelve el dataset completo (mayor peso), el movil un resumen ligero y el cajero solo el saldo numerico (menor peso). Esta verificacion se ejecuta automaticamente en las 95 pruebas del e2e.

5.3 Seguridad por canal

Cada BFF exige un scope OAuth2 especifico. Un token emitido para el canal web (scope=web) no puede acceder al BFF movil (scope=movil), y viceversa. El auth-server registra tres clientes independientes, uno por canal, cada uno con su propio secreto. La validacion del token se hace contra el JWKS publico del auth-server, sin llamadas de red adicionales.

5.4 Circuit Breaker (Resilience4j)

Cada BFF protege sus llamadas a cuentas y clientes con un Circuit Breaker de Resilience4j. Configure los parametros en el Config Server (config-repo):

Parametro	Valor	Efecto
slidingWindowSize	3	Evalua los ultimos 3 llamados
failureRateThreshold	50	Abre el circuito si >= 50% fallan
waitDurationInOpenState	10s	Tiempo en OPEN antes de pasar a HALF_OPEN
permittedNumberOfCallsInHalfOpenState	2	Llamados de prueba en HALF_OPEN

El fallback responde con HTTP 503 y un mensaje indicando que el servicio no esta disponible. Clientes es dato accesorio: si cae, web y movil responden igual pero sin el nombre del titular. Las transiciones del breaker (CLOSED → OPEN → HALF_OPEN → CLOSED) se verificaron en Docker con logs de nivel DEBUG.

6. Microservicios, Kafka y resiliencia

6.1 Saga de transferencias

Implemente una saga coreografiada sobre Kafka para las transferencias entre cuentas. Cada servicio reacciona a eventos sin un orquestador central: pagos inicia, cuentas ejecuta el debito y el credito, y si algo falla, cuentas compensa devolviendo el debito.

Flujo exitoso:

1. bff-web → POST /transferencias → pagos guarda como PENDIENTE
2. pagos → transferencia.iniciada → cuentas debita la cuenta origen
3. cuentas → transferencia.debito-realizado → cuentas acredita la cuenta destino
4. cuentas → transferencia.completada → pagos marca COMPLETADA, clientes notifica

Flujo con compensacion (cuenta destino inexistente):

- 1-3. Igual que arriba hasta el intento de credito
4. Credito falla → cuentas publica transferencia.credito-fallido
5. cuentas consume credito-fallido → devuelve el debito → transferencia.revertida
6. pagos marca REVERTIDA

6.2 Topics de Kafka

Topic	Publica	Consume	Significado
transferencia.iniciada	pagos	cuentas	Hay que debitar la cuenta origen
transferencia.debito-realizado	cuentas	cuentas	Debito OK, hay que acreditar destino
transferencia.debito-fallido	cuentas	pagos	Fondos insuficientes (FALLIDA)
transferencia.credito-fallido	cuentas	cuentas	Destino inexistente, compensar
transferencia.completada	cuentas	pagos, clientes	Transferencia exitosa
transferencia.revertida	cuentas	pagos	Compensacion aplicada (REVERTIDA)

Todos los topics tienen 3 particiones y replicacion 1. La key de cada mensaje es el transaccionId (UUID), garantizando que todos los eventos de una transferencia caigan en la misma particion y se procesen en orden. Cada servicio tiene su propio group-id (pagos-estado, cuentas-saga, clientes-notificaciones).

6.3 Idempotencia

Cuentas mantiene un EstadoSagaRepository (en memoria, con compare-and-set) que registra el paso de cada transferencia. Si Kafka reentrega un mensaje, el listener lo detecta y lo ignora. Pagos solo actualiza el estado de una transferencia que sigue en PENDIENTE, rechazando cambios duplicados.

7. Seguridad con Spring Cloud Security (OAuth2)

Implemente la seguridad con Spring Authorization Server 1.3.2, usando el flujo `client_credentials` que es el apropiado para comunicacion entre servicios (machine-to-machine).

7.1 Arquitectura de seguridad

- El **auth-server** emite tokens JWT firmados con RSA (clave generada al arrancar) y publica el JWKS en `/.well-known/jwks.json`. Los tokens tienen un TTL de 15 minutos.
- Cada **BFF** obtiene un token `client_credentials` con su propio cliente y scope, y lo adjunta a las llamadas hacia los microservicios backend.
- Los **microservicios** (cuentas, clientes, pagos) son Resource Servers OAuth2: validan la firma del JWT descargando el JWKS del auth-server y verifican el scope.
- El **healthcheck** de cuentas, clientes y pagos (`GET /actuador/health`) esta excluido de la seguridad para que Docker pueda verificar la salud sin token.

7.2 Clientes registrados

Cliente	Scope	Accede a
cliente-web	web	bff-web: cuentas completas, transacciones, transferencias
cliente-movil	movil	bff-movil: cuenta resumida
cliente-cajero	cajero	bff-cajero: saldo y retiro

7.3 Validacion cruzada

El e2e verifica que un token de un canal no pueda acceder a otro: un token con `scope=web` enviado al `bff-movil` recibe 403 Forbidden. Sin token, cualquier endpoint protegido devuelve 401. Esta validacion cubre los 6 servicios que actuan como Resource Server.

8. Contenerizacion y escalabilidad con Docker

8.1 Docker Compose

El archivo `docker-compose.yaml` define los 10 servicios Java, un broker Kafka y un volumen nombrado (`batch-data`) para los datos del batch. Cada servicio tiene:

- **Dockerfile multi-stage:** primera etapa compila con `maven:3.9-eclipse-temurin-21`, segunda etapa corre con `eclipse-temurin:21-jre-alpine` (~180 MB por imagen vs ~800 MB con JDK).
- **Healthcheck:** `curl` contra `/actuador/health` con `interval`, `timeout`, `retries` y `start_period` configurados. Los BFF usan un cliente HTTP interno (`HC_CLIENT`) porque su `/actuador/health` exige token.
- **depends_on con service_healthy:** garantiza el orden de arranque sin scripts de espera. Config Server arranca primero, luego Eureka, luego auth-server y kafka, y finalmente los servicios de negocio y BFF.
- **Perfil docker:** cada servicio activa el perfil Spring 'docker' via variable de entorno, que apunta las URLs a nombres de contenedor (`kafka:9092` en vez de `localhost:9092`).

8.2 Consumo de recursos

Medido con docker stats en reposo despues del e2e (10 contenedores): **3.94 GB de RAM total**. El servicio mas pesado es pagos (524 MB, por H2 + JPA + Kafka), el mas liviano es config-server (320 MB). Una instancia t3.large de AWS (8 GB) tiene margen suficiente.

8.3 Escalabilidad horizontal

Demostramos la escalabilidad horizontal levantando 2 replicas de bff-web detras de un nginx con round-robin. El override docker-compose.escala-bff.yaml usa deploy.replicas: 2 y !override (Compose >= 2.24) para reasignar puertos. Resultados:

- Eureka muestra exactamente 2 instancias UP de BFF-WEB, cada una con su propio hostname (contenedor Docker).
- 10 llamadas via nginx (localhost:8080): 5 atendidas por bff-web-1 y 5 por bff-web-2 (distribucion equilibrada con zone en el upstream).
- Ambas replicas validan el mismo token OAuth2 (firman contra el mismo JWKS).
- El e2e completo pasa despues de volver a 1 replica (sin residuos).

9. Evidencia de ejecucion

Toda la evidencia se encuentra en evidencia/eft_docker/final/ del repositorio. A continuacion listo las pruebas realizadas y sus resultados:

Prueba	Archivo de evidencia	Resultado
Stack completo (build + healthy)	stack_completo.txt	10/10 servicios healthy, perfil docker activo
E2E docker (2 corridas)	e2e_docker_1.txt, e2e_docker_2.txt	95/95 OK en ambas corridas
Recorrido de la saga	saga_recorrido_docker.txt	Transferencia exitosa + compensacion verificadas
Circuit Breaker	cb_debug_docker.txt	CLOSED->OPEN->HALF_OPEN->CLOSED, 13/13 checks
Batch (3 jobs)	batch_desde_repo.txt	3/3 jobs EXITO
Batch falla permanente	batch_pruebas_falla.txt	4 intentos, exit 1, RestartCount 3
Batch falla transitoria	batch_pruebas_falla.txt	Recuperacion al 2do intento, exit 0
Escala bff-web	escala_bff_web/	2 replicas, balanceo 5/5, token valido en ambas
Healthcheck sin token	healthcheck_sin_token.txt	cuentas/clientes/pagos healthy con auth-server caido
Clon limpio	clon_limpio.txt	95/95 OK desde clon nuevo, sin dependencias locales
Recursos y requisitos	recursos_y_requisitos.txt	3.94 GB RAM en reposo (10 contenedores)

10. Desafios y soluciones

10.1 Token invalido tras reinicio del auth-server

Al reiniciar el auth-server, este genera un nuevo par de claves RSA. Los tokens en cache de los BFF (firmados con la clave anterior) quedan invalidos, pero los Resource Servers siguen aceptandolos hasta que descargan el nuevo JWKS. Al reiniciar cuentas, este descarga las claves nuevas y rechaza el token cacheado del BFF, produciendo un 401 transitorio que dura hasta que el token vence (15 min). Documente este comportamiento como hallazgo (hallazgo_token_tras_reinicio_auth.txt) y lo reproduce de forma determinista. La solucion definitiva requiere persistir la clave RSA, pero esta fuera del alcance de la actividad.

10.2 Orden de arranque sin scripts de espera

En la primera version usaba scripts bash con sleep y curl para esperar a que los servicios estuvieran listos. Lo reemplazo por depends_on con condition: service_healthy, que es la forma nativa de Compose. El unico costo es que cada servicio necesita un healthcheck funcional, lo que resuelve exponiendo /actuator/health sin token en los microservicios backend.

10.3 Escalado con !override y replicas

El primer intento de escala uso `--scale` en la línea de comandos, pero un `docker compose up -d` posterior (sin el flag) reconciliaba el servicio a 1 replica, perdiendo la segunda. La solución fue declarar `deploy.replicas: 2` en el override YAML y usar `!override` para reasignar los puertos del host (8092-8093 en vez de 8081). Esto requiere Compose `>= 2.24`.

10.4 Eureka lease y nginx zone

En la primera prueba de escala, nginx distribuía 10/0 (todas las peticiones a una sola replica). El problema era que nginx asignaba 12 workers y sin la directiva `zone` el upstream no compartía estado entre ellos. Al agregar `zone` en el upstream, la distribución pasó a 5/5.

11. Propuestas de mejora

Las siguientes mejoras quedan fuera del alcance de esta EFT pero representan los pasos naturales para llevar este ecosistema a producción:

- **Base de datos persistente (PostgreSQL/RDS):** reemplazar H2 en memoria en cuentas y pagos por una base compartida. Esto permite escalar esos servicios horizontalmente y que los saldos sobrevivan reinicios.
- **Gestión de secretos:** mover los secretos OAuth2 y las credenciales de Kafka a AWS Secrets Manager o HashiCorp Vault, eliminándolos del config-repo.
- **Clave RSA persistente:** guardar el par de claves del auth-server en un volumen o en Secrets Manager para que un reinicio no invalide los tokens en curso.
- **HTTPS con ALB:** colocar un Application Load Balancer de AWS con certificado ACM delante de los BFF, terminando TLS en el balanceador.
- **Kafka gestionado (MSK):** reemplazar el broker standalone del compose por Amazon MSK para alta disponibilidad, replicación y monitoreo integrado.
- **Orquestación con ECS/EKS:** migrar de Docker Compose en una EC2 a ECS Fargate (o EKS) para auto-scaling, rolling updates y service mesh.

12. Conclusiones

El proyecto demuestra una migración completa del sistema legacy del Banco XYZ a una arquitectura moderna de microservicios. Los cinco ejes de migración se implementaron y verificaron con evidencia:

- **Batch:** 3 jobs con particionamiento, skip, retry y recuperación ante fallos, ejecutados en Docker con volumen nombrado y restart policy.
- **BFF:** 3 canales diferenciados en seguridad, contenido y peso de payload, con Circuit Breaker y fallback funcional.
- **Microservicios:** 3 servicios de negocio independientes con comunicación asíncrona vía Kafka y consistencia eventual garantizada por la saga.
- **Seguridad:** OAuth2 con Spring Authorization Server, JWT con RSA, scopes por canal y validación cruzada.
- **Docker:** 10 servicios contenerizados con builds multi-stage, healthchecks, arranque ordenado y escalabilidad horizontal demostrada.

Todo se verificó con una prueba automatizada de 95 verificaciones, ejecutada en un clon limpio del repositorio sin dependencias locales.